

Saner conda-forge interactions with a custom forge

Rohit Goswami · 2026-08-14 · packaging · conda · workflow

`conda-forge` is the destination. The docker loop is not how I get there.

Background

Last year I had a pile of packages that needed to exist as conda packages before they were allowed to exist on `conda-forge`. The Atomistic Cookbook recipe for eOn + PET-MAD is the one I still quote: I announced a conda build of eOn on **July 26 2025** because the cookbook needed it that day. The `eon-feedstock` did not land until **December**. The gap is in [the eOn write-up](#). The graduate students who tried the prefix channel in July could install the package. Everyone else waited on review.

`conda-smithy` locally wants Docker for the Linux builds. That loop did not work well for me. I was iterating recipes on a laptop I do not want to turn into a docker host, and I needed a channel I could point `pixi` or `micromamba` at the same afternoon.

Anyway, the thing that stuck is a small personal forge, same `recipe.yaml` shape as staged-recipes, built with `rattler-build`, uploaded to [prefix.dev](#).

Failed alternatives

`conda-smithy` + Docker

The blessed local path. Fine if you already live in that container. I did not. Rerender tricks for off-brand packages still live in the [smithy snippet](#).

Wait for staged-recipes

Correct destination. Wrong inner loop. eOn sat five months between “I need this for a cookbook” and “the feedstock exists”.

`mts-forge` as the ecosystem answer

The Metatensor playground that this layout started from. It is [archived](#). The ecosystem went to `conda-forge`. The forge idea is still useful for the leftovers.

`rg-forge`

The repo is [HaoZeke/rg-forge](#). The public channel is `https://repo.prefix.dev/rg-forge`. Recipes are `rattler-build v1` `recipe.yaml` files, one directory per package, plus a shared `conda_build_config.yaml`. `pixi` owns the builder:

```
git clone https://github.com/HaoZeke/rg-forge
cd rg-forge
pixi run build ira
```

That is `rattler-build build --experimental -m conda_build_config.yaml -m ira/variants.yaml --no-build-id --recipe ira`. `USE_SCCACHE=1` is a separate task when the compile is worth caching.

Upload is one command once the channel exists and `pixi auth prefix.dev` (or `PREFIX_API_KEY`) is set:

```
rattler-build upload prefix -c rg-forge output/linux-64/*.conda
```

Install from the other side:

```
micromamba install -c "https://repo.prefix.dev/rg-forge" -c conda-forge ira
# or, in pixi.toml
# channels = ["https://prefix.dev/rg-forge", "conda-forge"]
```

The July 2025 eOn line was the same channel:

```
micromamba install -c "https://repo.prefix.dev/rg-forge" eon
```

After the feedstock

When a package is on `conda-forge`, the forge recipe goes away. The eOn recipe was deleted on **2026-07-27**, after the feedstock was the real install path. `ira` now has [ira-feedstock](#) as well. What stays on `rg-forge` is the stuff that is not there, or not there in the form I need:

- `c-capnproto` and `capnp-janet`, the C / Janet runtimes I actually ship
- the `*-musl-static` variants, Zig-built archives under `$PREFIX/musl-static/<name>/` so they do not collide with host shared libs
- a `vesin` recipe when I need one that is not [python-vesin](#)

CI is a Nickel file that writes `.github/workflows/build.yml`. `linux-64`, `osx-64`, `osx-arm64`, `linux-aarch64`. `win-64` is still commented out; that is the same `windows-latest` / VS 18 mess as in [the Windows post](#). One broken recipe is `continue-on-error` so a bad `ira` build does not block uploading `c-capnproto`.

Caveats

A personal forge is not a feedstock.

- There is no bot. Compiler pins drift. I keep `conda_build_config.yaml` by hand and I am the one who notices when `conda-forge` moved.
- Split builds are worse than Azure. The matrix is whatever GitHub runners I listed, not the full forge.
- `nushell` was a bad fit for the build scripts. It would not pick up the `pixi` Python without a native file. The recipes that matter use `build.sh`.
- `prefix.dev` can host a private channel. I do not put secrets in the public one. Auth stays in `pixi auth` / the CI secret, not in the recipe.

The destination is still `conda-forge`: official source, one package per staged-recipes PR, feedstock PRs from a fork. The forge is how I find out the recipe works before I ask someone else to review it.

Conclusions

When the feedstock exists, I delete the forge recipe and point `pixi` at `conda-forge`.

